

pif_eth_100 — Development Report

Date: 2026-09-30. Generated at 2026-09-30T14:24Z from the session transcripts, with a transcript cut-off of 2026-09-30T14:24:52.015Z. **The session was still running when this report was generated:** final verification (Task 14), the final whole-branch review and the finishing of the branch come after this snapshot, so their tokens and time are not in any total below.

All figures are simulator measurements, not silicon claims.

1. Summary

- **Delivered:** a self-contained `source/pif_eth_100/` variant of `pif_eth`: PRU0 sends 200 B BERT frames (xorshift32 payload plus CRC-32 FCS on the hardware CRC widget, 8b/10b coded) at TXCFG `n = 3` on a 300 MHz core, and PRU1 receives them over the perif loopback at exactly 2x oversampling (fractional RX divider 1.5). Plus golden-model tests, a scripted UI walkthrough, a user's guide, a README, this report and PDFs of the guide and the report.
- **Throughput (simulator):** F1 line rate 100.00 Mbaud; F2 in-burst data rate 80.00 Mbit/s (the 8b/10b coding rate; the payload alone in a burst, F2p, is 76.05 Mbit/s); F3 frame-averaged goodput 53.18 Mbit/s TX-limited and 8.97 Mbit/s end-to-end with the baseline RX. Zero BER, `crc_ok = 1`, `rx_ovf = 0` and 0 symbol errors on all 18 sweep runs (7 seeds; all three sample-to-edge phases for the default seed).
- **Fast RX (an extra deliverable):** `pif_eth_100_rx_fast.asm` cuts the RX post-frame work from 44 469 to 14 905 PRU1 cycles (3-frame mean, 148.2 us to 49.7 us) with results bit-identical to the baseline, and raises end-to-end goodput from 8.97 to 19.48 Mbit/s (21.25 Mbit/s in steady state).
- **Caveats:** the 0 ns loopback latency failed one edge phase, so the harness uses a 5/6 ns modelled wire latency (section 9); the UI guide was validated by script, not in a real browser (section 11).
- **Effort (snapshot):** planning on Opus 5.5, implementation and task reviews on Sonnet 5.5. Coordinator-reported subagent tokens: 1,895,582 (section 5, measure A). Transcript-derived total: 68,490,234 tokens, of which 65,592,397 are cache reads and 2,897,837 are not (measure B). Elapsed time from the first prompt to the cut-off: 3 h 05 min (section 6).

2. The request

The user's original prompt, verbatim (typos as written):

the pif_eth project in the source folder goes up to 125 Mbit with `n=2`. I need another variant with 8b/10b line coder CRC32 ussing the hardware widget and 200 bytes frame with 80 Mbit net data rate and 100 Mbit line rate. PRU

frequency should be 300 MHz to make $N=3$. Use superpower planing with opus 5.5 and implementation with Sonnet 5.5. Put the project in a separate folder e.g. pif_eth_100. Document whole development in a report including porompt, interaction, tokens, models, time spent. Also generate a users guide on how to run the receive and transmitter in the UI.

3. Decisions and interaction log

3.1 Decisions

Source: design spec §1 and the §17 addendum (docs/superpowers/specs/2026-09-30-pif-eth-100-design.md).

Topic	Decision
"100 Mbit line rate"	100 Mbaud: TXCFG $n = 3$ at a 300 MHz core, 10 ns per line bit.
"80 Mbit net data rate"	Interpreted as the 8b/10b coding rate: 100 Mbaud $\times 8/10 = 80$ Mbit/s of octets while a burst is on the wire (F2). Commas, FCS and the inter-frame gap lower the frame-averaged goodput (F3); that is measured and reported next to it, not hidden.
Folder	Separate, self-contained source/pif_eth_100/; source/pif_eth/ is not modified.
TX base	pif_eth_tx_n2.asm (all FIFO-full checks kept), copied; only the TXCFG word changes to 0x00020010 ($n = 3$). FCS on the hardware CRC widget (pif_eth_crc32_hw.inc, copied).
Frame	BERT frame, 200 B xorshift32 payload + 4 B FCS = 204 octets before 8b/10b, bracketed by K28.5 commas.
RX	RX Option 1 (realtime raw capture, decode after the frame) on PRU1, over the existing zero-drift perif loopback channel 0. RX divider 1.5 (div_factor 0, FRAC = 1, RXCFG 0x0000801F) gives exactly 2x oversampling (5 ns samples).
Core clock	300 MHz on both cores (config/memory_pif_eth_100.cfg); the root memory.cfg is never used by the tests and only changed transiently by the UI run.
Loopback latency rule (spec 17.2)	Default 0 ns; all three sample-to-edge residues are tested by shifting the latency by 0, 10/3 and 20/3 ns. A fixed 5/6 ns latency is only the fallback if the 0 ns setting fails on the edge-tie signature (second gate in plan Task 5). The user chose "keep the plan: try 0 ns, then fall back". Outcome: the second gate in Task 5 triggered. At 0 ns the edge-phase test at the 10/3 ns shift failed (symbol_errors = 71, crc_ok = 0); with 5/6 ns all three phases were clean, so LOOPBACK_LATENCY_NS = 5/6 ns.
RX optimisation (spec 17.1)	A separate, post-baseline deliverable, pif_eth_100_rx_fast.asm (fast aligned decode with a 2:1 decimation LUT). The baseline pif_eth_100_rx.asm stays untouched; if the fast RX cannot pass, the baseline remains the delivered result.
Existing files (spec 17.3)	Only a changelog entry under "Unreleased" in the top-level readme.md and a new docs/handoff/ note; no other existing file changes.
UI validation (spec 17.4)	A scripted walkthrough over the server's HTTP/WS endpoints is the validation of record; a real-browser run is added only if the Chrome extension answers.
Branch and commits (spec 17.5)	Branch feat/pif-eth-100 off feat/sd-sweep; one commit per task; explicit-path staging only; no push, no PR; commit trailer naming Sonnet 5.5.

3.2 Interaction log

Times are UTC (session start 2026-09-30T13:19:49+02:00 = 11:19:49Z). This log is a sanitised summary: system reminders, memory contents and tool output are deliberately not reproduced.

This log is hand-supplemented. `session_stats.py` deliberately drops `tool_result` records, and the answers to the coordinator's multiple-choice questions arrive in the transcript as tool results, so the tool's own "Interaction log" output would show only the plain-text user messages. The main session has 4 multiple-choice question rounds (with answers) and 5 plain-text user messages (the original prompt, "yes, go ahead", "approve, Sonnet 5.5 subagent-driven", and two "continue with the plan" resumes). The other user-role records with text are relayed subagent messages and task notifications, and are not listed. The rows below combine both sources; question rounds are marked (question).

Time (UTC)	Who	What
11:19:44	User	The original prompt (section 2). Classified as an architectural task (new firmware variant plus documentation), so: brainstorming, spec, plan, execution.
11:20:44	User (question 1, "Frame size")	Chose "200 B payload + 4 B FCS (Recommended)", that is 204 octets before 8b/10b, with 80 Mbit/s read as the 8b/10b coding rate. Effect: the frame is a 200 B PRNG payload plus FCS; the goodput is measured, not assumed.
11:23:17	User (question 2, "RX sampling")	Chose "2x oversample via fractional divider 1.5 (Recommended)"; the alternatives offered were 3x at $n_{rx} = 1$ and 1x at $n_{rx} = 3$. Effect: the old rule $n_{rx} = n_{tx} / 2$ cannot work at $n_{tx} = 3$, and the fractional divider gives exactly 2x.
11:24:46	User	"yes, go ahead" (approve the design; start the spec).
11:25:13 - 11:37:51	Planning subagent (Opus 5.5)	Wrote the design spec (12.6 min).
12:03:29	User (question 3, "Spec review", "Goodput", "Extra files", "Commits")	Spec: "Approve as written". Goodput (end-to-end about 9 Mbit/s because RX decodes after the frame): "Also add an RX-optimisation task". Extra files: "Handoff note + readme entry (Recommended)". Commits: "Work on a new branch, commit per task (Recommended)". Effect: the spec addendum (§17) adds a separate optimised RX (<code>pif_eth_100_rx_fast.asm</code>), a readme changelog entry and handoff note, and branch <code>feat/pif-eth-100</code> with one commit per task.
12:03:56 - 12:57:37	Planning subagent (Opus 5.5)	Spec addendum §17 and the implementation plan (53.7 min elapsed, including a ~10 min gap inside the run).
13:22:53	User (question 4, "Plan / exec", "Latency")	Not an approval at first: the user asked to be sure that the transmission on the wire is 100 Mbit, and said that the throughput table in the plan was not clear. Latency question: "Keep the plan: try 0 ns, then fall back (Recommended)". The coordinator then explained that the wire stays 100 Mbit/s (80 Mbit/s of data inside a burst), and that the table's goodput is 1600 payload bits divided by the whole frame period (burst 21.04 us + TX prep about 9 us + RX

Time (UTC)	Who	What
		post-frame decode about 148 us at baseline), which is why the averages are lower.
13:24:58	User	"approve, Sonnet 5.5 subagent-driven": one Sonnet 5.5 subagent per task, each followed by a Sonnet 5.5 task reviewer; Opus 5.5 for the final whole-branch review.
13:25 onward	Coordinator and Sonnet 5.5 subagents	Implementation tasks 2 to 10, each with an implementer and a reviewer (see the per-agent table in section 5). At Task 5 the loopback-latency second gate triggered (section 3.1); the fallback of 5/6 ns was used. The Chrome extension was not connected (two attempts in Task 8), so the UI guide was validated by script only. The coordinator ran Task 1 (branch and commit) itself.
13:54:09 - 13:59:41	Coordinator (tooling pause)	The harness's auto-mode safety classifier returned "no verdict (error)" for every Bash and Agent call (nine in a row: six before the user's first "continue", three after it). The coordinator stopped after two retry rounds and told the user; the user's two "continue with the plan" messages below followed. The shell worked again at 13:59:41 and the Task 10 reviewer was launched at 13:59:57. No work was lost or redone; the pause is inside Task 10's wall-clock in section 6.
13:56:51	User	"continue with the plan": execution resumed after a transient outage of the harness safety classifier (Bash and Agent calls returned "no verdict"); the coordinator had stopped after two retry rounds and told the user. No work was lost.
13:59:38	User	"continue with the plan" (second resume, same cause).
13:59:57 - 14:03:11	Sonnet 5.5 reviewer and the original Task 10 implementer	The Task 10 review found that this section had omitted all four question rounds, had called the plain-text messages "three" (there are five) and had placed the latency question in the wrong round. One fix round (the same implementer resumed), then a clean re-review.
14:03:29 - 14:10:05	Coordinator and Sonnet 5.5 subagents	Task 11 (optimised RX, section 8): implementer and reviewer; the coordinator independently re-ran <code>run_100.py --rx both</code> (18 of 18 PASS) and got the same numbers.
14:10:21 - 14:12:57	Coordinator and Sonnet 5.5 subagents	Task 12 (changelog entry in <code>readme.md</code> , handoff note): implementer and reviewer, clean.
14:13:41 onward	Coordinator and the Task 13 subagent (Sonnet 5.5)	Final report, <code>build_pdfs.py</code> , PDFs, and one consolidated documentation pass over README, user's guide, changelog wording and handoff note, which folded in the documentation findings that reviewers had recorded on Tasks 2 to 12 (section 9). No user message arrived after 13:59:38.
after the cut-off	(pending)	Task 14 final verification, the final whole-branch review (Opus 5.5, planned, not yet run at generation time) and the finishing of the branch.

4. Process and phases

1. Brainstorming (main session, Sonnet 5.5): interpret the request, agree the terms in the spec §1 table.

2. Design spec (Opus 5.5 planning subagent): 12.6 min; reported by the coordinator as 209,015 subagent tokens and 43 tool uses.
3. User review of the spec, with the decisions recorded as the §17 addendum.
4. Spec addendum and implementation plan (Opus 5.5, the same planning agent): 53.7 min; the coordinator reported 461,515 subagent tokens and 69 tool uses for the second run, which the coordinator treated as cumulative across both planning runs.
5. Implementation: one Sonnet 5.5 subagent per plan task, each followed by a spec and code-quality review subagent (Sonnet 5.5). Task 1 (branch and commit) was run by the coordinator.
6. Final verification (Task 14), final whole-branch review (Opus 5.5) and finishing the branch: pending at this snapshot.

Plan tasks and their commits (`git log --oneline feat/sd-sweep..HEAD` at generation time, plus this report's own commit):

Task	Content	Commit
1	Branch, design spec and plan	a8a6af1, 83a4753 (plan uses the session scratchpad instead of /tmp)
2 (T0)	300 MHz config, golden-model copies, fractional-divider spike	1151b86
3 (T1)	frames.py BERT-200 frame builder, golden-model tests	fe0cf2c
4 (T2)	PRU0 TX at n = 3 and the run_100 driver core	ee0cf1d
5 (T3)	PRU1 RX at 2x via divider 1.5, traced loopback run	b5b9603
6 (T4)	Measured F1/F2/F3 throughput and seed-sweep CLI	c791be1
7 (T5)	seed_ui_100.py seed/arm/status for the browser UI	808d97b
8 (T6)	Scripted UI walkthrough and the user's guide	e5f5444
9 (T7)	README with clock plan, maps and measured throughput	295ee3f
10 (T8)	session_stats.py and the report skeleton	2753584, fix round c431c94
11	Optimised RX (pif_eth_100_rx_fast.asm), equivalence tests	7aef7c6
12	Changelog entry in readme.md, handoff note	6f4b458
13	This report, build_pdfs.py, the two PDFs, consolidated documentation pass	the commit that contains this file
14	Final verification	pending

5. Models and tokens

Generated 2026-09-30T14:24Z; transcript cut-off 2026-09-30T14:24:52.015Z (UTC); session start 2026-09-30T13:19:49+02:00 (first prompt 11:19:44Z). The Task 13 agent (the one producing this report) and the coordinator were still running, so both appear with partial figures, and Task 14 and the final review are not included.

Models. Planning: `claude-opus-5-5` (Opus 5.5; the model id is the `model` field of the transcript records and the `resolvedModel` of the launch record). Implementation and task reviews: `claude-sonnet-5-5` (the `sonnet` alias resolved to this id in every launch record). The coordinator session itself is `claude-sonnet-5-5`. Final whole-branch review: Opus 5.5, planned; not run at generation time.

Two different token measures appear in this report. **They are not comparable and must not be added.**

Measure A: coordinator-reported subagent tokens

The completion summary that the subagent tool hands to the coordinator (`subagent_tokens`, `tool_uses`, `duration_ms`). It is a much smaller accounting whose basis the tool does not document, and it cannot be recomputed from the transcripts. Where an agent was resumed (the planner, and the Task 10 implementer) two summaries exist and are shown as "first / second"; the second figure is the last one reported and is what the sums use, on the assumption (not stated by the tool) that it is cumulative. "Active minutes" is the sum of the runs' `duration_ms`, so it excludes waiting between two runs.

Agent	Task (launch description)	Role	Model (resolved at launch)	Reported tokens	Tool uses	Active minutes
afef5f3cea2c8b370	Opus writes pif_eth_100 spec	planner	claude-opus-5-5	209,015 / 461,515	43 / 69	12.6 / 53.7
a9eff8cf3c249a707	Implement Task 2: T0 spike gate	implementer	claude-sonnet-5-5	54,434	9	1.1
ab26866df04b4299e	Review Task 2 (spec + quality)	reviewer	claude-sonnet-5-5	65,462	5	0.6
aaf1c15487d56c01f	Implement Task 3: frames.py + tests	implementer	claude-sonnet-5-5	41,363	10	0.6
abf0616162471a3c1	Review Task 3 (spec + quality)	reviewer	claude-sonnet-5-5	42,098	5	0.4
a807a88e6106c1f8c	Implement Task 4: TX fw + run_100 core	implementer	claude-sonnet-5-5	55,677	10	1.2
ad8b188c10aa5609c	Review Task 4 (spec + quality)	reviewer	claude-sonnet-5-5	63,074	7	0.5
a53ed3425580afb81	Implement Task 5: RX baseline + loopback	implementer	claude-sonnet-5-5	51,107	12	1.3

Agent	Task (launch description)	Role	Model (resolved at launch)	Reported tokens	Tool uses	Active minutes
ab61557445a453093	Review Task 5 (spec + quality)	reviewer	claude-sonnet-5-5	62,371	6	0.9
a0c720c095cbfba77	Implement Task 6: throughput + sweep CLI	implementer	claude-sonnet-5-5	50,035	9	1.4
a3c1113f677fe8585	Review Task 6 (spec + quality)	reviewer	claude-sonnet-5-5	51,208	6	0.9
a4e793ab2d015dc48	Implement Task 7: seed_ui_100.py	implementer	claude-sonnet-5-5	49,543	11	1.4
acd4da7d0b0b3eb90	Review Task 7 (spec + quality)	reviewer	claude-sonnet-5-5	53,410	11	0.8
af6fae04e443e69f5	Implement Task 8: UI walkthrough + guide	implementer	claude-sonnet-5-5	94,499	26	3.2
a2f1645e52054199b	Review Task 8 (spec + quality)	reviewer	claude-sonnet-5-5	62,233	9	1.2
aa7053c0722b5afe4	Implement Task 9: README.md	implementer	claude-sonnet-5-5	72,982	20	2.8
a4f74912584baf7d	Review Task 9 (spec + quality)	reviewer	claude-sonnet-5-5	59,339	11	1.2
a0693ee6db1b999ae	Implement Task 10: session_stats + report	implementer	claude-sonnet-5-5	102,661 / 114,607	23 / 8	4.4 / 1.0
a860b28982b310de7	Review Task 10 (spec + quality)	reviewer	claude-sonnet-5-5	66,963	9	1.2
af105b62010070d1d	Re-review Task 10 fix round 1	reviewer	claude-sonnet-5-5	38,841	3	0.3
a0a0fc0dcc5318f95	Implement Task 11: optimised RX	implementer	claude-sonnet-5-5	84,477	30	4.7
ad40770356525c249	Review Task 11 (spec + quality)	reviewer	claude-sonnet-5-5	79,553	8	1.2

Agent	Task (launch description)	Role	Model (resolved at launch)	Reported tokens	Tool uses	Active minutes
a34706eb92e791b4e	Implement Task 12: readme + handoff	implementer	claude-sonnet-5-5	62,213	14	1.4
a523ebc70484c3ec4	Review Task 12 (spec + quality)	reviewer	claude-sonnet-5-5	58,578	10	1.0
a58a4f5ea9de40de5	Implement Task 13: report, PDFs, doc pass	implementer	claude-sonnet-5-5	not yet reported	not yet reported	not yet reported

Sum of last reported figure	Tokens
Planner (Opus 5.5)	461,515
11 implementer agents, Tasks 2 to 12 (Sonnet 5.5; Task 13's is not yet reported)	730,937
12 reviewer agents (Sonnet 5.5)	703,130
All reported	1,895,582

Active minutes: planner 66.3, implementers 24.4, reviewers 10.2; 384 tool uses in all reported runs.

Measure B: transcript-derived, de-duplicated

Method: every assistant record with `message.usage` is de-duplicated by `message.id` (a streamed reply is written as several records that repeat the usage; per id the maximum of each counter is kept), then input, cache-write, cache-read and output tokens are summed. **Cache reads are shown separately because they inflate the totals:** every API call re-reads the whole conversation prefix from the cache, so the "Total" column grows with the number of calls times the context size, not with the amount of new work. The last column removes them.

Caveat: the `output_tokens` recorded for many agents look very small (for example a few hundred for agents that wrote whole files), so the output counts are probably lower bounds; this was not investigated.

Model	Input	Cache write	Cache read	Output	Total	Total without cache reads
claude-opus-5-5	190	1,147,557	24,893,289	30,731	26,071,767	1,178,478
claude-sonnet-5-5	858	1,597,458	40,699,108	121,043	42,418,467	1,719,359

Agent	Role	Model	Start (UTC)	Wall-clock (min)	Total	of which cache read	Total without cache reads
main (coordinator)	coordinator	claude-sonnet-5-5	11:13:46	180.0	21,720,078	21,380,235	339,843
afef5f3cea2c8b370	planner	claude-opus-5-5	11:25:13	92.4	26,071,767	24,893,289	1,178,478
a9eff8cf3c249a707	implementer	claude-sonnet-5-5	13:25:56	1.1	374,680	315,102	59,578
ab26866df04b4299e	reviewer	claude-sonnet-5-5	13:27:19	0.6	154,375	108,276	46,099
aaf1c15487d56c01f	implementer	claude-sonnet-5-5	13:28:10	0.6	222,336	198,617	23,719
abf0616162471a3c1	reviewer	claude-sonnet-5-5	13:29:03	0.4	67,516	45,553	21,963
a807a88e6106c1f8c	implementer	claude-sonnet-5-5	13:29:47	1.2	371,301	333,081	38,220
ad8b188c10aa5609c	reviewer	claude-sonnet-5-5	13:31:09	0.5	253,432	208,994	44,438
a53ed3425580afb81	implementer	claude-sonnet-5-5	13:31:51	1.3	436,422	402,874	33,548
ab61557445a453093	reviewer	claude-sonnet-5-5	13:33:23	0.9	179,395	139,873	39,522
a0c720c095cbfba77	implementer	claude-sonnet-5-5	13:34:32	1.4	334,998	302,524	32,474
a3c1113f677fe8585	reviewer	claude-sonnet-5-5	13:36:05	0.9	208,002	172,925	35,077
a4e793ab2d015dc48	implementer	claude-sonnet-5-5	13:37:10	1.4	379,835	347,685	32,150
acd4da7d0b0b3eb90	reviewer	claude-sonnet-5-5	13:38:44	0.8	304,573	270,144	34,429
af6fae04e443e69f5	implementer	claude-sonnet-5-5	13:39:54	3.2	1,510,114	1,395,867	114,247

Agent	Role	Model	Start (UTC)	Wall-clock (min)	Total	of which cache read	Total without cache reads
a2f1645e52054199b	reviewer	claude-sonnet-5-5	13:43:21	1.2	405,781	359,832	45,949
aa7053c0722b5afe4	implementer	claude-sonnet-5-5	13:44:49	2.8	878,386	822,961	55,425
a4f74912584baf7d	reviewer	claude-sonnet-5-5	13:47:49	1.2	419,975	379,988	39,987
a0693ee6db1b999ae	implementer	claude-sonnet-5-5	13:49:23	13.3	2,242,992	2,061,000	181,992
a860b28982b310de7	reviewer	claude-sonnet-5-5	13:59:57	1.2	310,704	246,818	63,886
af105b62010070d1d	reviewer	claude-sonnet-5-5	14:02:54	0.3	102,942	82,672	20,270
a0a0fc0dcc5318f95	implementer	claude-sonnet-5-5	14:03:29	4.7	1,727,827	1,660,990	66,837
ad40770356525c249	reviewer	claude-sonnet-5-5	14:08:55	1.2	364,186	299,502	64,684
a34706eb92e791b4e	implementer	claude-sonnet-5-5	14:10:21	1.4	494,872	445,004	49,868
a523ebc70484c3ec4	reviewer	claude-sonnet-5-5	14:11:59	1.0	277,791	240,698	37,093
a58a4f5ea9de40de5	implementer	claude-sonnet-5-5	14:13:41	11.2	8,675,954	8,477,893	198,061

The main (coordinator) row's wall-clock, 180.0 min, runs from the main transcript's first record to its last; the first records (session setup from 11:13:46Z) precede the first prompt (11:19:44Z), so it overstates the working time slightly.

Why A and B differ. For the planner, measure A says 209,015 (spec run) and 461,515 (after the plan); measure B says 26,071,767 in total, of which 24,893,289 are cache reads. A is the harness's own summary figure; B is a sum over every API call of the counters in the transcript, where the same cached context is counted again on every call. Neither is a bill and neither is more "true"; A is the one to quote as "subagent tokens", B shows how much context was processed and how much of it was cache reads.

6. Time spent

Session start 2026-09-30T13:19:49+02:00 (11:19:49Z); first prompt 11:19:44Z; transcript cut-off 2026-09-30T14:24:52.015Z; elapsed at the cut-off: 3 h 05 min (a snapshot; the session was still running). Times are UTC.

Phase	Start	End	Wall-clock
Brainstorming (main session)	11:19:44	11:25:13	about 5.5 min
Design spec (Opus 5.5)	11:25:13	11:37:51	12.6 min
Spec review with the user	11:37:51	12:03:56	about 26 min
Spec addendum and plan (Opus 5.5)	12:03:56	12:57:37	53.7 min
Plan finished until approval	12:57:37	13:24:58	about 27 min
Implementation, Tasks 2 to 13	13:25:56	14:24:52 (snapshot)	see the table below

Implementation per task (implementer start to the last reviewer end; the coordinator's work between agents is included):

Task (implementer start to last reviewer end)	Start (UTC)	End (UTC)	Wall-clock
Task 2 (T0 spike)	13:25:56	13:27:57	2.0 min
Task 3 (T1 frames)	13:28:10	13:29:28	1.3 min
Task 4 (T2 TX)	13:29:47	13:31:37	1.8 min
Task 5 (T3 RX baseline)	13:31:51	13:34:19	2.5 min
Task 6 (T4 throughput)	13:34:32	13:36:58	2.4 min
Task 7 (T5 seed_ui)	13:37:10	13:39:34	2.4 min
Task 8 (T6 UI walkthrough + guide)	13:39:54	13:44:31	4.6 min
Task 9 (T7 README)	13:44:49	13:49:01	4.2 min
Task 10 (T8 stats + report skeleton), incl. tooling pause and fix round	13:49:23	14:03:11	13.8 min
Task 11 (RX-fast)	14:03:29	14:10:05	6.6 min
Task 12 (readme + handoff)	14:10:21	14:12:57	2.6 min
Task 13 (this task: final report, PDFs, doc pass), still running at the cut-off	14:13:41	14:24:52 (snapshot)	11.2 min so far

The 11 implementer agents of Tasks 2 to 12 were active for 24.4 min in total and the 12 reviewer agents for 10.2 min (sums of the reported `duration_ms`; the Task 13 agent has no reported figure yet, so it is in neither sum). Task 10's window includes the tooling pause (section 3.2) and the fix round; the per-agent wall-clock of a resumed agent (first to last record) also contains the waiting between its runs, which is why the Task 10 implementer's transcript wall-clock in section 5 is longer than its reported active time. Final verification, the final whole-branch review and finishing are not in these figures.

7. Results

All results are from simulator runs on 2026-09-30 (commit 6f4b458 plus the uncommitted Task 13 documentation edits, which do not touch any firmware or test-visible code); a run made in Task 13 (the task that produced this report) is marked "(this task)". Nothing here is a silicon measurement. Reproduce with the commands given.

7.1 Fractional-divider spike (this task)

`python3 source/pif_eth_100/spike_frac_rx.py` runs the unmodified `pif_eth` firmware at 300 MHz (TXCFG overridden to `0x00020010`, RX control word `0x0000801F`):

```
PASS S0.1 cores at 300 MHz pru0=300.0 pru1=300.0
PASS S0.2 fractional RX divider 1.5 rx_period=5.0 ns tx_period=10.0 ns
PASS S0.3 TX n=3 @300MHz seed=464371934 pushed=263 T_burst=21040.000000ns invalid=
0
PASS S0.4 python-armed RX lat=0.0000 seed=464371934 cap=525B invalid=0 edge_ties=1
266
PASS S0.4 python-armed RX lat=3.3333 seed=464371934 cap=525B invalid=0 edge_ties=0
PASS S0.5 firmware RX lat=0.0000 seed=464371934 hot_loop=8cyc/byte max_rx_fifo=1 l
ast_stats=[2, 526, 0, 0, 1, 0, 1600, 1]
SPIKE PASS: 23/23 checks, base latency 0.0 ns
```

(Six of the 23 lines are shown; the other 17 are the same checks for the other seeds and shifts, all PASS.) The spike passed at a base latency of 0.0 ns, and the loopback harness later failed at the same base latency at one shift: the two harnesses arm differently and place the edge-tie phase at different shifts (README "Latency note"); the loopback test is the gate of record.

7.2 Loopback sweep and throughput (this task)

`python3 source/pif_eth_100/run_100.py --rx both` (seed 464371934 plus seeds 1 to 6, 3 frames each; the default seed at three latencies for the three edge phases), verbatim:

```
TX only (seed 464371934, 4 back-to-back frames): PASS period=10.000 ns min transit
ion spacing=10.000 ns on 10 ns grid=True burst_pushed=[263, 263, 263, 263]
```

rx	seed	lat_ns	frm	ovf	symerr	biterr	crc	eof	cap	fifo	hot	post_cyc	res
ult													
base	464371934	0.833	3	0	0	0	1	1	526	1	8	44469	PAS
S													
base	464371934	4.167	3	0	0	0	1	1	526	1	8	44469	PAS
S													
base	464371934	7.500	3	0	0	0	1	1	526	1	8	44469	PAS
S													
base	1	0.833	3	0	0	0	1	1	526	1	8	44481	PAS
S													
base	2	0.833	3	0	0	0	1	1	526	1	8	44457	PAS
S													
base	3	0.833	3	0	0	0	1	1	526	1	8	44462	PAS
S													
base	4	0.833	3	0	0	0	1	1	526	1	8	44508	PAS
S													
base	5	0.833	3	0	0	0	1	1	526	1	8	44470	PAS

```

S
base          6  0.833  3  0  0  0  1  1  526  1  8  44445  PAS
S
fast 464371934 0.833  3  0  0  0  1  1  526  1  8  14904  PAS
S
fast 464371934 4.167  3  0  0  0  1  1  526  1  8  14904  PAS
S
fast 464371934 7.500  3  0  0  0  1  1  526  1  8  14904  PAS
S
fast          1  0.833  3  0  0  0  1  1  526  1  8  14916  PAS
S
fast          2  0.833  3  0  0  0  1  1  526  1  8  14892  PAS
S
fast          3  0.833  3  0  0  0  1  1  526  1  8  14898  PAS
S
fast          4  0.833  3  0  0  0  1  1  526  1  8  14943  PAS
S
fast          5  0.833  3  0  0  0  1  1  526  1  8  14906  PAS
S
fast          6  0.833  3  0  0  0  1  1  526  1  8  14880  PAS
S

```

Throughput [base RX] (simulator figures, not silicon)

F1 line rate 100.00 Mbaud

F2 in-burst data rate 80.00 Mbit/s (8b/10b coding rate)

F2p in-burst payload rate 76.05 Mbit/s (200 B per 21.04 us burst)

F3 goodput, TX-limited 53.18 Mbit/s (gap 9.05 us)

F3 goodput, end-to-end 8.97 Mbit/s (gap 157.35 us)

T_burst 21040.0 ns RX post-frame 44469 cycles = 148.2 us host poll bound <= 160 ns

Throughput [fast RX] (simulator figures, not silicon)

F1 line rate 100.00 Mbaud

F2 in-burst data rate 80.00 Mbit/s (8b/10b coding rate)

F2p in-burst payload rate 76.05 Mbit/s (200 B per 21.04 us burst)

F3 goodput, TX-limited 53.18 Mbit/s (gap 9.05 us)

F3 goodput, end-to-end 19.48 Mbit/s (gap 61.09 us)

T_burst 21040.0 ns RX post-frame 14905 cycles = 49.7 us host poll bound <= 160 ns

OVERALL: PASS

Figure	Baseline RX	Fast RX
F1 line rate	100.00 Mbaud	100.00 Mbaud
F2 in-burst data rate (8b/10b coding rate)	80.00 Mbit/s	80.00 Mbit/s
F2p in-burst payload rate (200 B per 21.04 us)	76.05 Mbit/s	76.05 Mbit/s
F3 goodput, TX-limited	53.18 Mbit/s (gap 9.05 us)	53.18 Mbit/s (gap 9.05 us)
F3 goodput, end-to-end, 3 frames	8.97 Mbit/s (gap 157.35 us)	19.48 Mbit/s (gap 61.09 us)
RX post-frame, 3-frame mean	44469 cycles = 148.2 us	14905 cycles = 49.7 us

T_burst = 21040.0 ns (263 FIFO bytes x 8 x 10 ns); host poll bound at most 160 ns. All 18 rows PASS with hot = 8 (the realtime capture loop costs 8 PRU1 cycles per byte against a budget of 12), fifo 1, cap 526.

7.3 Loopback latency gate (this task)

The 0 ns evidence, with the command to reproduce it (README "Latency note"): the default seed at latency 0.0 + shift for the three edge-phase shifts gave clean, **not clean** (symbol_errors=71, crc_ok=0, bit_err=547, cap_bytes=525) and clean, respectively, at 0.0000, 3.3333 and 6.6667 ns; the fast RX fails identically at 3.3333 ns. At the harness's 5/6 ns base latency, seeds 1, 2 and 3 at all three shifts (2 frames each) gave 9 of 9 clean.

7.4 Scripted UI walkthrough (this task)

python3 source/pif_eth_100/ui_walkthrough_100.py --frames 2 against python3 ui/server.py --port 8091 with config/memory_pif_eth_100.cfg copied to memory.cfg (private port; 8080 was not touched; memory.cfg restored with git checkout -- memory.cfg and compared byte-identical afterwards). Run at 14:16 UTC in the order base, fast, base, fast, all exit 0:

RX	Frame 1	Frame 2	Stats after each frame
baseline (pif_eth_100_rx.asm)	PASS after 35000 lead instructions	PASS after a further 35000	frames 1 then 2, cap_bytes 526, rx_ovf 0, symbol_errors 0, crc_ok 1, bit_err 0, tot_bits 1600, eof_status 1
fast (pif_eth_100_rx_fast.asm)	PASS after 20000	PASS after a further 15000	same fields, same values

(The instruction counts are the multiples of 5000 at which the script checked, so they are upper bounds.) After the 2-frame baseline run, read_memory at 0x2E00 returned a 204-byte frame whose first eight bytes are b0a1abf161f49863 (the start of frame 2's xorshift32 payload) and whose FCS is ed97f06d. This is a script over the server's HTTP/WebSocket endpoints, **not a real-browser click-through** (section 11).

7.5 Tests (this task)

```
python3 -m pytest tests/test_pif_eth_100.py    57 passed
python3 -m pytest tests/test_pif_eth.py        60 passed
python3 -m pytest tests/test_pif_eth_rx.py     15 passed
```

(132 passed in a single combined run.) The new test in this task, test_build_pdfs_renders_gfm_tables, failed first with ImportError: cannot import name 'build_pdfs' and passed once build_pdfs.py existed.

8. RX optimisation

Outcome: delivered. `pif_eth_100_rx_fast.asm` passes every mandatory check (zero BER, `crc_ok = 1`, `rx_ovf = 0`, `symbol_errors = 0`) and gives frame bytes and stats identical to the baseline. The baseline `pif_eth_100_rx.asm` is unchanged and remains the proven reference. It was built only after Tasks 1 to 10 (baseline loopback, tests, UI guide, report skeleton) were committed, as the user decided. Simulator results only, not silicon.

Why the baseline is slow. The baseline `post_frame` visits the 8 samples of each captured byte one at a time, about 20 cycles per line bit, and takes about 44.5 k PRU1 cycles (148 us at 300 MHz) per 200 B frame. The realtime capture loop is not the bottleneck (8 cycles per byte, RX FIFO depth 1). Because frame `i+1` may only start after PRU1 has finished frame `i`, end-to-end goodput is 8.97 Mbit/s against 53.18 Mbit/s TX-limited.

Approach. A fast aligned decode, after the first comma has anchored the symbol grid:

1. On the first frame's post-frame only, PRU1 builds a 256 B 2:1 decimation LUT (`nib[b] = b7<<3 | b5<<2 | b3<<1 | b1`, the samples the baseline's XOR toggle keeps) at local `0x0C00`. It is not built at boot: in the UI both go flags are set before either core runs, so RX must arm within TX's frame prep (about 2.7 k cycles), and a boot-time build in the planner's scratch prototype missed the frame start (`cap_bytes 410`, `crc_ok 0`).
2. The baseline bit-slide scan runs unchanged until the first comma, then at the next byte boundary `pf_fast` loads 4 capture bytes per `lbbo`, maps each byte to 4 line bits with one LUT load, and cuts a 10-bit symbol whenever at least 10 bits are held. Decode, running disparity, commas and the frame-buffer guard follow `pf_symbol`'s aligned branch.

Why Options 2 and 3 were rejected. The 2026-07-21 RX spec's Options 2 and 3 move work into the realtime loop and were estimated at 15 to 25 cycles per byte, which does not fit the 12-cycle per-byte budget (8 samples at 1.5 core cycles). The chosen approach leaves the realtime loop untouched: `diff` of the `poll:` to `eof:` region against the baseline prints nothing.

Prototype evidence (planner, not the implementer). Before this task the planner ran the plan's code in scratch copies: first on the 250 MHz rig (`n_tx = 4`) with 4 seeds x payloads 128/200/252 x 3 latencies, identical stats and frame bytes (post-frame 29.2 k to 9.0 k, 44.5 k to 13.5 k, 55.5 k to 16.8 k cycles); then the exact plan code at 300 MHz (13.5 k cycles steady state, F3 end-to-end 19.48 Mbit/s). Those are the hypotheses Task 11 re-measured; the numbers below are Task 11's own runs, and Task 13 re-ran `run_100.py --rx both` and got identical figures (section 7.2).

Measured before/after at 300 MHz (`run_100.py --rx both`, seed 464371934, 3 frames, latency 0.8333 ns, traced single-stepped PRU1 cycles eof to frame_loop; all 18 rows PASS with `hot = 8`, OVERALL: PASS):

Quantity	Baseline RX	Fast RX
Post-frame, 3-frame mean (incl. one-off LUT build)	44 469 cycles = 148.2 us	14 905 cycles = 49.7 us

Quantity	Baseline RX	Fast RX
Post-frame per frame	44 484 / 44 444 / 44 480	17 652 / 13 513 / 13 549
Post-frame, steady state (frames 2 and 3)	about 44 460 cycles	about 13 531 cycles = 45.1 us
F3 end-to-end, 3 frames	8.97 Mbit/s	19.48 Mbit/s
F3 end-to-end, steady state (frames 2 to 5 of a 5-frame run)	8.97 Mbit/s	21.25 Mbit/s
F3 TX-limited	53.18 Mbit/s	53.18 Mbit/s

The post-frame time falls by 66 % over 3 frames and by 70 % in steady state; the one-off LUT build costs the first frame about 4.1 k cycles (17 652 against 13 531). These match the planner's prototype figures to within the reported rounding. End-to-end goodput stays below the TX-limited 53.18 Mbit/s because the remaining gap is 61.09 us (3-frame) against 9.05 us TX-limited.

Equivalence tests (`tests/test_pif_eth_100.py`, 14 new, all `rx="fast"`): LUT empty at boot and equal to the expected table after frame 1; UI flow with both go flags set at boot is clean; 3-frame loopback clean and `FrameStats` list equal to the baseline's; equal to the baseline for 2 seeds x 3 edge phases and for payload lengths 128 and 252; realtime loop unchanged (hot deltas, 8 cycles per byte, RX FIFO depth at most 2); post-frame at least 2x faster; F3 end-to-end at least 1.5x the baseline and below the TX-limited figure. The TDD RED run failed with `FileNotFoundError` for the missing `pif_eth_100_rx_fast.asm`; the first run with the brief's code was green. These tests use clean frames only; the invalid-codeword, running-disparity-violation and frame-buffer-overflow branches of the fast decode are not covered.

Scripted UI check. `ui_walkthrough_100.py --frames 2 --rx fast` passed on a private port (8091), as did a base re-load and a fast re-load (each with Reset); the fast frames were done after 20 000 and 15 000 lead instructions against 35 000 for the baseline. As in section 9, this is a script over the server's endpoints, not a browser click-through.

9. Deviations from the spec and plan

- **Loopback latency fallback (taken).** Spec 17.2 anticipated it, and the user had chosen "try 0 ns, then fall back". The first gate (the spike, Task 2) passed at 0.0 ns with 23 of 23 checks, so `LOOPBACK_LATENCY_NS` was provisionally 0.0. The second gate (Task 5, the edge-phase loopback test at the 10/3 ns shift) failed at 0 ns (`symbol_errors 71, crc_ok 0`); the fallback of 5/6 ns was used, documented as a modelled wire propagation delay, not drift. With it all three edge phases and seeds 1 to 3 decode clean; the decoder is still not robust at an exact sample/edge tie (section 10, section 11).
- **The coordinator ran Task 1 itself.** The branch and the first commit are exact git commands with no judgement, so the coordinator did them instead of dispatching a subagent.
- **Plan `/tmp` paths replaced by the session scratchpad.** The environment's rule for temporary files replaced every `/tmp/...` path of the plan (commit 83a4753);

content unchanged.

- **--rx guard in run_100.py (Task 6).** `main()` calls `ap.error` when the firmware file for the chosen `--rx` mode is missing; approved as harmless.
- **The fast-RX guide step is 4c** (step 4b already existed), and one assembler comment in the fast RX was changed from "built at boot" to "built on the first frame" (Task 11).
- **UI validation by script only.** The Chrome extension was not connected (two attempts in Task 8), so the user's guide was validated through the scripted walkthrough over the server's HTTP/WS endpoints (spec 17.4 fallback), not in a real browser. Two implementers (Tasks 8 and 10) reported `DONE_WITH_CONCERNS`.
- **What reviewers changed.** Every task review from Task 2 to Task 12 came back clean except Task 10, which needed one fix round (the interaction log, section 3.2). The other findings were minor and were carried forward as a list of documentation fixes and applied in Task 13 in one pass: README (evidence cited from committed files and a reproducible command instead of gitignored notes; the 9.05 us TX gap worded as consistent with the spec's estimate rather than measured; the `fifo` and `hot` columns explained; the spike-at-0-ns versus loopback-at-0-ns difference reconciled; plan jargon removed; why goodput is 19.5 and not 80 Mbit/s even with the fast RX; fast-RX equivalence tests cover clean frames only); user's guide (a "derived from the UI source, not clicked" notice; a shorter intro; 8080 as the default port; two numbers with no saved evidence checked or removed, see the next item; the 71-symbol-error claim cited to its test); top-level `readme.md` changelog (the 5/6 ns caveat added to the zero-BER claim; the one pre-existing repository file that the branch changes, apart from the transient `memory.cfg`); the handoff note ("Implementation complete" instead of "Complete").
- **A wrong number in the guide found and fixed in Task 13.** The guide said the first eight bytes of the reconstructed frame observed were `b0a1abf161f49863` without saying which frame. Re-reading the frame after a 2-frame walkthrough reproduced it, and the bytes belong to frame 2 (the PRNG stream continues across frames; frame 1 starts `a521c86ebe053c0c` per the golden model). The guide now says so. The "10 000 / 40 000 lead instructions" claim had no saved evidence and was removed.
- **A tooling outage** (section 3.2) paused execution for about 5.5 minutes; not a deviation from the plan, but it is inside Task 10's wall-clock.
- **Review findings not addressed** (minor, listed in section 11), because the files they concern were outside what this task could change.

10. Lessons learned

- **Edge ties and floating point.** At 300 MHz the sample-to-edge offset takes one of three values, {0, 1.667, 3.333} ns, so at 0 ns wire delay samples land exactly on TX edges in one of the three arm phases. The 300 MHz core period (10/3 ns) is inexact in floating point, so exact ties resolve inconsistently once an inexact shift is added; the loopback test at the 10/3 ns shift showed 71 symbol errors. A 5/6 ns

modelled wire delay keeps every phase at least 0.83 ns from an edge. One frame passing in the UI at 0 ns is no evidence that 0 ns is safe.

- **A spike harness can pass a check that the production harness fails.** The 0 ns latency passed in the spike and failed in `run_100`, because arming differs and the tie phase lands at a different shift. Cover every residue by construction, not by the single base case, and treat the production-harness test as the gate of record.
- **Never poll `pc == label` between multi-instruction steps.** A 2-instruction wait loop can alias with the polling step and never match. The fix was the traced stepper (`run_100.step_paced_traced`), which mirrors the simulator's paced stepping but reports every PRU1 instruction; it also must not step PRU1 alone, since that would run it ahead of PRU0's time.
- **Measure, do not hand-tally.** Cycle counts near a hard limit (the 12-cycle per-byte budget of the realtime RX loop, 8 measured) were confirmed by single-stepping the simulator, and the same goes for the post-frame cycles. The planner's prototype figures for the fast RX matched the implementer's re-measurement to within rounding, and this task's re-run of `run_100.py --rx` both matched again.
- **Do not build the fast RX's LUT at boot.** In the UI both go flags are set before either core runs, so RX must arm within TX's frame preparation (about 2.7 k cycles); a boot-time LUT build missed the frame start (`cap_bytes 410, crc_ok 0`). The first frame's post-frame builds it instead.
- **Loading a program keeps the core's old PC and registers,** so a Reset is needed after loading (spec 17.4); skipping it produced stats full of garbage.
- **Token totals need care.** Transcript totals need de-duplication by message id and a clear separation of cache reads; otherwise they are inflated several times over and are not comparable with a coordinator's reported subagent tokens (section 5). Streaming also seems to leave `output_tokens` small in the transcripts.
- **An interaction log needs both sources.** The tool that summarises transcripts drops tool results, and the answers to multiple-choice questions arrive as tool results, so a machine-made log silently omits them; the review caught the omission.
- **Documentation may only cite what a reader can see.** Evidence in gitignored notes or a scratch directory is not evidence for a repository reader; a reviewer caught such citations and they were replaced by a committed test and a runnable command. Numbers with no saved evidence should be re-verified or dropped (one turned out to belong to a different frame than the text implied).
- **Scripted validation is not a browser click-through.** State it wherever the UI wording is used; the guide says so at the top.
- **Keep a progress ledger.** When the harness's safety classifier failed mid-task, the recorded state let the run resume exactly where it stopped, with nothing lost or repeated.

11. Caveats

- **Simulator only.** No figure here has been measured on silicon.
- **Fractional divider on silicon.** The RX divider 1.5 is exact in the simulator (an even 5.000 ns sample spacing). On silicon the sample edges may be spaced

unevenly (1 and 2 core clocks alternating); that would still give 2 samples per bit at zero drift but less edge margin. This is not verified against the TRM (spec risk R6).

- **DRAM timing** in the simulator assumes `write_latency = 1` and `jitter = 0`.
- **The 0 ns edge-tie limit remains.** RX decode is not robust when a sample edge coincides exactly with a TX edge; nothing was done to make it robust at 0 ns, and the harness sidesteps it with 5/6 ns.
- **No real-browser click-through was performed.** The user's guide was validated by a scripted walkthrough over the server's endpoints; the button labels, dropdown names and the Project flow come from reading the UI source. The Chrome extension was not connected.
- **The fast RX equivalence tests exercise clean frames only.** The invalid-codeword, running-disparity-violation and frame-buffer-overflow branches of the fast decode have no test.
- **F3 is still bounded** by the RX post-frame work and the TX preparation; the end-to-end figures are 8.97 Mbit/s (baseline RX) and 19.48 Mbit/s (fast RX), against the 80.00 Mbit/s that holds only while a burst is on the wire.
- **Token, model and time tables are a snapshot** of a session that was still running (Task 14, the final whole-branch review and the finishing of the branch come after it). Token totals include cache reads (section 5); per-agent wall-clock is first to last record; the recorded output-token counts look low.
- **The interaction log is a sanitised, hand-supplemented summary**, not a transcript.
- **Minor review findings left open** (all outside the files this task could change, none affecting the results): the spike's hot-loop metric passes on an empty measurement, and its edge-tie counts are printed, not asserted; F2 is close to a tautology and the F3 values are pinned only by ordering in the tests; reviewers noted that `run_100.py --frames 1` and an empty `--seeds` list fail with a traceback or crash instead of a message (not re-checked); the edge-phase test covers only the default seed with one frame per shift; the fast RX asm carries a second header block; the "realtime loop is byte-identical" check (`diff`) is not a committed test.